

University of Chittagong
Department of Computer Science and Engineering
7th Semester B.Sc. (Engg) Examination-2024
Course Code: CSE-711 **Course Title: Compilers**
 Total marks: 54 Marks Time: 4.00 hours

[Answer any *three* questions from each of the *Section-A* and *Section-B*.
 Figures in the right-hand margin indicate full marks]

Section-A

1. a) Define a compiler and identify the different types of compilers. 1.5
- b) Provide a diagram illustrating the components and workflow of a language-processing system. 1.5
- c) List the phases of a compiler and explain each phase in detail. Also specify the output produced by every phase. 6
2. a) How does a parser play an important role in compiler design? 2
- b) Explain how regular expressions can be converted into context-free grammar by considering the regular expression $(a \mid b)^*abb$ 3
- c) Explain the connection among tokens, pattern and lexemes. 2
- d) Differentiate between scanning and parsing. Name some popular tools used as lexer and parser. 2
3. a) Explain the functions of input buffer and transition diagram in compilers. 2
- b) What is syntax tree? Construct the syntax tree for the following:
 $position = initial + rate * 70;$ 2
- c) Narrate the Leftmost and Rightmost Derivations. Consider the grammar:
 $E \rightarrow E + E \mid E * E \mid (E) \mid id$ 4
 Find out leftmost and rightmost derivations of $(id+id)*id$.
- d) Demonstrate the ambiguity of a grammar with clear example. 1
4. a) State the parsing in compiler. Compare and contrast top-down parsing and bottom-up parsing with illustration. 2.5
- b) Show the behaviors of Recursive Descent Parser and its error recovery. 2.5
- c) What is Left Recursion? How to eliminate it? Eliminate left recursion from the following grammar.
 $E \rightarrow E+T \mid E-T \mid E\%T \mid T \mid id$ 3
- d) Given the following grammar, find out the terminals, non-terminals and start-symbol.
 $S \rightarrow D$
 $D \rightarrow T \mid id \mid A;$
 $T \rightarrow int \mid float$
 $A \rightarrow = \mid num \mid \epsilon$ 1.5

Section-B

5. a) Consider the following grammar: 3
 $E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow (E) \mid \text{id}$
Find out the FIRST and FOLLOW of all possible non-terminals.
- b) Construct the parse table of above grammar. 3
- c) Illustrate the parsing of Input-string $(\text{id} + \text{id}) * \text{id}$ using Predictive Parsing. 3
6. a) Compare the LR(0), SLR, LR(1) and LALR Parsers. 1
- b) Consider the grammar in question 5.a) Demonstrate the parsing of input string $(\text{id} + \text{id}) * \text{id}$ using Shift-Reduce Parsing. 3
- c) What are synthesized and inherited attributes? What are S-attributed and L-attributed grammars? 2
- d) From the following grammar build the Syntax Directed Definition (SDD) and design the annotated parse tree for $3 * 5 + 4n$. 3
 $L \rightarrow E n$
 $E \rightarrow E_1 + T$
 $E \rightarrow T$
 $T \rightarrow T_1 * F$
 $T \rightarrow F$
 $F \rightarrow (E)$
 $F \rightarrow \text{digit}$
7. a) Define synthesized attribute and inherited attribute with illustrative examples. 2
- b) Explain the notion of Syntax Directed Definition (SDD). 1
- c) Define basic block and flow graph with example. 2
- d) What are the differences between syntax tree and Directed Acyclic Graph (DAG)? 4
Draw the DAG for the expression:
 $a + a * (b - c) + (b - c) * d$
8. a) How to translate an arithmetic expression into i) three-address code, ii) quadruples, iii) triples, and iv) indirect-triples 4
- b) From the given program (computation of dot product) show the three-address statements performing this computation. 3
begin
 $\text{prod} := 0;$
 $i := 1;$
 do begin
 $\text{prod} := \text{prod} + a[i] * b[i];$
 $i := i + 1$
 end
while $i \leq 20$
end
- c) Calculate the instruction cost of the given instruction sequence. 2
MOV b, R0
ADD c, R0
MOV R0, a.

Department of Computer Science and Engineering
University of Chittagong
7TH Semester BSc (Engg.) Examination 2023
CSE 711: COMPILERS
Marks: 54 Time: 4 hours

[Using section-wise separate answer scripts, answer any three questions from each of the following two sections and all the parts of a question chronologically.]

SECTION A

- A-1/ (a) Write regular definitions for the following language: 2
 i. unsigned numbers such as 3234, 2334.34, 454.34E5, or 334.34E - 4.
 ii. all strings of lowercase letters that contain the five vowels in order
 (b) What tasks are performed in the *front-end* and *back-end* of a compiler, and why? 3
 (c) What is the role of symbol tables in compilers? Describe the components of a typical symbol table and outline its implementation using a hash table approach in C. 4
- A-2/ (a) Consider the following grammar: 4

$bepr \rightarrow bepr \text{ or } bterm \mid bterm$
 $bterm \rightarrow bterm \text{ and } bfactor \mid bfactor$
 $bfactor \rightarrow \text{not } bfactor \mid (bepr) \mid \text{true} \mid \text{false}$

- i. Left factor this grammar if necessary.
 ii. Does left factoring make the grammar suitable for top-down parsing?
 iii. In addition to left factoring, eliminate left recursion from the original grammar.
 iv. Is the resulting grammar suitable for top-down parsing?
 (b) How recursive-descent parsing works? Construct a **recursive-descent** parser for the following grammar. 5

$S \rightarrow \text{if } C \text{ then } S$
 $\quad \mid \text{while } C \text{ do } S$
 $\quad \mid \text{id} = \text{num} ;$
 $\quad \mid \text{id} ++ ;$
 $C \rightarrow \text{id} == \text{num} \mid \text{id} != \text{num}$

- A-3/ (a) Consider the following grammar

$S \rightarrow \text{if } (E) \text{ then } (E) X \mid E$
 $X \rightarrow \epsilon \mid \text{else } (E)$
 $E \rightarrow \text{true } Y \mid \text{false } Y$
 $Y \rightarrow \epsilon \mid \text{and } (E)$

where S is the start symbol and {if, then, else, and, true, false, (,)} is the set of terminals. Compute the associated **FIRST** and **FOLLOW** sets. Then construct the **LL(1)** parsing table for the grammar (put only the production numbers in the table). 6

- (b) Briefly explain the error recovery strategy in **predictive** parsing. 3
- A-4/ (a) Differentiate between scanning and parsing. Name some popular tools used as lexer and parser. 2
 (b) The following grammar for **if-then-else** statements is proposed to remedy the dangling else ambiguity 5

$stmt \rightarrow \text{if } exp \text{ then } stmt$
 $\quad \mid \text{matched_stmt}$
 $\text{matched_stmt} \rightarrow \text{if } exp \text{ then matched_stmt else stmt}$
 $\quad \mid \text{other}$

Show that this grammar is still ambiguous.

- (c) Explain how a regular expression can be converted into a context-free-grammar by considering the regular expression $(a \mid b)^*abb$. 2

SECTION B

- B-1/ (a) What are the different type of conflicts can occur in **shift-reduce** parsing? 2
 (b) Consider the following grammars and associated semantic actions.

$G \rightarrow F$	$G.p = F.p$
$F \rightarrow F_1 \wedge F_2$	$F.p = \text{And}(F_1.p, F_2.p)$
$F \rightarrow F_1 \vee F_2$	$F.p = \text{Or}(F_1.p, F_2.p)$
$F \rightarrow \neg F_1$	$F.p = \text{Neg}(F_1.p)$
$F \rightarrow F_1 \Rightarrow F_2$	$F.p = \text{Or}(\text{Not}(F_1.p), F_2.p)$
$F \rightarrow (F_1)$	$F.p = F_1.p$
$F \rightarrow id$	$F.p = id.lexeme$

- Say whether each attribute of a non-terminal is inherited or synthesized and why. 2
 - Show the annotated parse tree for the expression $\neg(A \wedge (A \Rightarrow B))$ 2
 - Write a yacc/bison program to generate a parser for the given grammar. 3
- B-2/ (a) Translated the expression $-(a+b)*(c+d) + (a+b+c)$ into i) quadruples, ii) triples, iii) indirect triples. 3
- (b) Consider the following code segments that count the number of primes from 2 to n using the sieve method on a suitably large array a . Translate the program in to three address statements. Assume integers require 4 bytes. 4

```

for (i=2; i<=n; i++)
    a[i] = TRUE;
count = 0;
s = sqrt(n);
for (i=2; i<=s; i++)
    if (a[i]) /* i has been found to be a prime*/
        count++;
for (j=2*i; j<=n; j = j+1)
    a[j] = FALSE; /* no multiple of i is a prime*/

```

- (c) Define copy propagation. What is the advantage of copy propagation? 2
- B-3/ (a) Upon determining the basic blocks, construct a control-flow graph corresponding to the following three-address code. 2.5

(1) $i = m - 1$	(16) $t_7 = 4 * i$
(2) $j = n$	(17) $t_8 = 4 * j$
(3) $t_1 = 4 * n$	(18) $t_9 = a[t_8]$
(4) $v = a[t_1]$	(19) $a[t_7] = t_9$
(5) $i = i + 1$	(20) $t_{10} = 4 * j$
(6) $t_2 = 4 * i$	(21) $a[t_{10}] = x$
(7) $t_3 = a[t_2]$	(22) goto(5)
(8) if ($t_3 < v$) goto (5)	(23) $t_{11} = 4 * i$
(9) $j = j - 1$	(24) $x = a[t_{11}]$
(10) $t_4 = 4 * j$	(25) $t_{12} = 4 * i$
(11) $t_5 = a[t_4]$	(26) $t_{13} = 4 * n$
(12) if ($t_5 > v$) goto (9)	(27) $t_{14} = a[t_{13}]$
(13) if ($i >= j$) goto (23)	(28) $a[t_{12}] = t_{14}$
(14) $t_6 = 4 * i$	(29) $t_{15} = 4 * n$
(15) $x = a[t_6]$	(30) $a[t_{15}] = x$

- (b) What are the objectives of code optimization? Apply, in stages, all valid and relevant optimizing transformations to the flow graph obtained in B-3(a) 5
- (c) Construct the DAG for the basic block 1.5

```

(1)  $d = b * c$ 
(2)  $e = a + b$ 
(3)  $b = b * c$ 
(4)  $a = e - d$ 

```

- B-4/ (a) Consider the following C code that compute Fibonacci numbers recursively. Suppose that the activation record for f includes the following elements in order: (return value, argument n , local s , local t) Assume the initial call is $f(5)$

```

int f(int n) {
    int t, s;
    if (n < 2) return 1;
    s = f(n-1);
    t = f(n-2);
    return s+t;
}

```

- Show the complete activation tree. 3
 - What does the stack and its activation records look like the first time $f(1)$ is about to return? [hint. snapshot] 3
- (b) Differentiate between static and dynamic storage allocation. What is the role of the garbage collector? 3

Department of Computer Science and Engineering
University of Chittagong
7th Semester B.Sc. Engineering Exam 2022
CSE 711: COMPILERS
Marks: 52.5 Time: 4 hours

[Answer any three of the four questions from each of the two sections below. Answers to parts of a question should appear contiguously in the answer script. Ensure readability of your handwriting.]

SECTION A

- A-1/ (a) Certain aspects of computer architecture design relate to elements of compiler design and implementation in significant ways. Identify them and briefly explain the connection.
(b) What tasks are performed in the *front-end* and *back-end* of a compiler, and why? How are the *phases* related to the *passes*?
(c) What advantages are there to a language-processing system in which the compiler produces assembly language rather than machine language?
(d) *Symbol table is a necessary component of a compiler* – justify this statement with an example. [2+3+2+2]

- A-2/ (a) Briefly explain the error recovery strategies in syntax analysis.
(b) Introduce the notions of *token*, *pattern* and *lexeme*. Point out how they are related.
(c) Describe the language denoted by the following regular expression and provide transition diagrams to recognize the language: $(a|b)^*abb(a|b)$
(d) How does the lexical analyzer read the input string and break it into lexemes. [3+2+3+1]

- A-3/ (a) Let G be a Context Free Grammar for which the production rules are given below:

$$\begin{aligned} S &\rightarrow aB \mid bA \\ A &\rightarrow a \mid aS \mid bAA \\ B &\rightarrow b \mid bS \mid aBB \end{aligned}$$

Derive the string $aaabbabbba$ using the above grammar (using Left Most Derivation and Right Most Derivation).

- (b) Why is left recursion not a problem for bottom-up parsing?
(c) Consider the following grammar and eliminate left recursion

$$X \rightarrow XSb \mid Sa \mid b \quad S \rightarrow Sb \mid Xa \mid a$$

- (d) Left factor the following grammar:

$$A \rightarrow ABd \mid Aa \mid a \quad B \rightarrow Be \mid b$$

- A-4/ (a) How does recursive descent parser work? Describe with an example.
(b) Compute the FIRST and FOLLOW for the given CFG:

$$\begin{aligned} S &\rightarrow aXZh & X &\rightarrow cY & Y &\rightarrow bY \mid \epsilon \\ Z &\rightarrow EF & E &\rightarrow g \mid \epsilon & F &\rightarrow f \mid \epsilon \end{aligned}$$

- (c) For the grammar below, a partial LL(1) parsing table is presented along with the grammar. Entries that need to be filled are indicated as E_1 , E_2 , and E_3 . [3+3+3]

$$\begin{aligned} S &\rightarrow aAbB \mid bAaB \mid \epsilon \\ A &\rightarrow S \\ B &\rightarrow S \end{aligned}$$

	a	b	\$
S	E_1	E_2	$S \rightarrow \epsilon$
A	$A \rightarrow S$	$A \rightarrow S$	error
B	$B \rightarrow S$	$B \rightarrow S$	E_3

SECTION B

B-1/ (a) What is the difference between LR parser and LL parser? Explain your answer.

(b) Verify that following grammar is LL(1) or not?

$$S \rightarrow Ab\$ \quad A \rightarrow (bAb) \quad A \rightarrow (Ab) \quad A \rightarrow \lambda$$

(c) On input (id) * id, give the sequence of stack and input contents for the following parsing table.

Productions		Action					GoTo	
		*	(	)	id	\$	T	F
0			S5		S8		2	1
1	$T \rightarrow F$	R1	R1	R1	R1	R1		
2	$T \rightarrow T * F$	S3				Acc		
3	$F \rightarrow id$		S5		S8			4
4	$F \rightarrow (T)$	R2	R2	R2	R2	R2		
5			S5		S8		6	1
6		S3		S7				
7		R4	R4	R4	R4	R4		
4		R3	R3	R3	R3	R3		

[2+4+3]

B-2/ (a) Construct an LR parsing table for the given context-free grammar:

$$S \rightarrow AA \quad A \rightarrow aA \mid b$$

(b) Differentiate between inherited and synthesized attributes with examples.

(c) Draw a dependency graph for the expression: $3 * 5 * 2$. Production rules are given below:

Production	Semantic Rules
1) $T \rightarrow FT'$	$T'.inh = F.val$ $T.val = T'.syn$
2) $T' \rightarrow * FT_1'$	$T_1'.inh = T'.inh \times F.val$ $T'.syn = T_1'.syn$
3) $T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4) $F \rightarrow digit$	$F.val = digit.lerval$

[5+2+2]

B-3/ (a) What is the difference between L-attributed and S-attributed grammar? Is the following grammar L-attributed or not? Explain your answer.

PRODUCTION	SEMANTIC RULES
$A \rightarrow BC$	$A.s = B.b$ $B.i = f(C.c, A.s)$

(b) Generate code for the following three-address statements assuming a and b are arrays whose elements are 4-byte values: $x = a[i] \quad y = b[i] \quad z = x * y$

(c) Write quadruples, triples and indirect triples for the expression:

[3+3+3]

$$-(a * b) + (c + d) - (a + b + c + d)$$

B-4/ (a) Consider the following program in 3-address intermediate code

(1) $a = 2$	(7) $r_1 = t_1 + r_1$	(13) $r_2 = t_2 + r_2$
(2) $b = 3$	(8) goto (5)	(14) goto (11)
(3) $t = a$	(9) $t = b$	(15) $t_3 = r_1 + r_2$
(4) $r_1 = 0$	(10) $r_2 = 0$	(16) $r = 8 * t_3$
(5) if ($t \leq 0$) goto (9)	(11) if ($t \leq 0$) goto (15)	
(6) $t_1 = 2 * a$	(12) $t_2 = 7 * b$	

i. Indicate where new basic blocks start. For each basic block, give the line number such that the instruction in the line is the first one of that block.

ii. Give names $B_1, B_2, \dots$ for the program's basic blocks in the order the blocks appear in the given listing. Draw the control flow graph making use of those names. Don't put in the code into the nodes of the flow graph, the labels B_i are good enough.

(b) What is code optimization? Explain machine dependent and independent code optimization.

[3+3+3]



Using section-wise separate answer-scripts, answer any three of the four questions from each of the following two sections. Write legibly.

SECTION A

Question 1: (3+3+1)+1.75 marks

- (a) Consider the following grammar, with terminal symbols *let*, *in*, *ident*, *:=*, and *\$*:

Now answer the followings:

- Write the FIRST and FOLLOW sets for the grammar.
- Compute the LL(1) parse table for the grammar.
- Is the grammar LL(1)? Why or why not?

$$T \rightarrow S\$$$

$$S \rightarrow D \text{ in } E$$

$$D \rightarrow V D \mid \epsilon$$

$$V \rightarrow \text{ident} := E \mid \text{let ident} := E$$

$$E \rightarrow \text{ident} \mid \epsilon$$

- (b) Define and differentiate between compile time errors and runtime.

Question 2: 2+(1+2+2+1.75) marks

- (a) How does a parser plays an important role in a compiler design?

- (b) The following is a grammar for regular expressions over symbols *a* and *b* only, using *+* in place of *|* for union, to avoid conflict with the use of vertical bar as a metasympol in grammars:

Now answer the followings:

- Left factor this grammar.
- Does left factoring make the grammar suitable for top-down parsing?
- In addition to left factoring, eliminate left recursion from the original grammar.
- Is the resulting grammar suitable for top-down parsing?

$$\text{rexpr} \rightarrow \text{rexpr} + \text{rterm} \mid \text{rterm}$$

$$\text{rterm} \rightarrow \text{rterm} \text{ rfactor} \mid \text{rfactor}$$

$$\text{rfactor} \rightarrow \text{rfactor} * \mid \text{rprimary}$$

$$\text{rprimary} \rightarrow a \mid b$$

Question 3: 3+3+2.75 marks

- (a) Introduce the notions of *token*, *pattern* and *lexeme*. Point out how they are related.

- (b) Pertaining to the principles of programming languages, clarify the following two distinctions: i) *environment* and *state*, ii) *dynamic* and *static*.

- (c) What is the function of lexical analyzer? Explain the reason for separating the analysis phase of compiling into lexical analysis and parsing.

Question 4: 2.75+3+3 marks

Consider the following grammar

$$S \rightarrow S a \mid b \mid \text{error } a$$

Now answer the followings.

- Construct the SLR sets of items for the grammar.
- Show the PDA or transition table for recognizing viable prefixes of this grammar.
- Compute GOTO function for these sets of items.

SECTION B

Question 5: 2+2.75+2+2 marks

- (a) Define synthesized attribute and inherited attribute with illustrative examples.

- (b) Construct a syntax-directed translation scheme that translates arithmetic expressions from postfix notation into infix notation.

- (c) Give annotated parse trees for the inputs $9 - 5 + 2$ and $9 - 5 * 2$.

- (d) What are S-attributed and L-attributed grammars?

Question 6: 4.75+4 marks

(a) Consider the following grammar and associated parsing table.

Show the execution of the parser on the string *abbc*.

(b) Translate the arithmetic expression $a * - (b + c)$ into i) a syntax tree and ii) three address code

(c) What is the intermediate code representation for the following expression: *a or b and not c*

0: $S \rightarrow X$
1: $X \rightarrow a X c$
2: $X \rightarrow X X$
3: $X \rightarrow b$

State	a	b	c	\$	S	X
1	s2	s3			g5	g4
2	s2	s3				g6
3	r3	r3	r3	r3		
4	s2	s3		r0		g7
5				Acpt		
6		s3	s8			g7
7	r2	r2	r2	r2		
8	r1	r1	r1	r1		

Question 7: 3+3+2.75 marks

(a) Reconstruct the C function that got compiled into the following keywords: *else, for, do, while*. the following x86-64 assembly.

```
fA:
movq    (%rdi), %rax
cmpq    %rsi, %rax
jl      .L1
testq   %rax, %rax
jne     .L3
.L1:
ret
.L3:
movq    %rsi, (%rdi)
jmp     .L1
```

The function takes two arguments that are passed through *%rdi* and *%rsi* respectively, as is the convention, and likewise for the return value.

(b) For the (not-very-meaningful-but-syntactically-correct) C function presented, produce its operationally equivalent C code that is free from compound logical operations (i.e., no *||* or *&&*) and does not use any of

```
long fn(long p, long q, long r, long n)
{
    long v;
    if(p < 0 && q < 0)
        v = 0;
    else if (p+q >= r)
        v = p+q;
    else v = (p>q)? p : q;

    for(int i=1; i<=n; i++)
    {
        if(p > q+i || q > p+i)
            v += (p-i)*(q-i);
    }
    return v;
}
```

(c) What is the role of *stack frames* (a.k.a. *activation records* or *call stack*)? How do the local variables of a function get represented in the associated frame? What are *caller saved* and *callee saved* registers?

Question 8: 1.75+3+4 marks

(a) Why compilers use intermediate representations instead of translating directly from source to target language?

(b) Draw the control flow graph for the following program:

```
test:
    cjmp (x < 0) done
    x = x - y
    y = y + 1
    jump test
done:
    z = y
```

(c) Given the following section of a three-address code for the bubble-sort algorithm, construct the associated flow-graph. Apply all the applicable optimizations techniques and show the evolution of the basic block through several optimization stages.

```
t8 ← j - 1
t9 ← 4 * t8
temp ← A[t9]
t10 ← j + 1
t11 ← t10 - 1
t12 ← 4 * t11
t13 ← A[t12]
t14 ← j - 1
t15 ← 4 * t14
A[t15] ← t13
t16 ← j + 1
t17 ← t16 - 1
t18 ← 4 * t17
A[t18] ← temp
```


University of Chittagong
Department of Computer Science and Engineering
7th Semester B.Sc. Engineering Examination 2020
CSE 711: Compilers
Marks: 52.5 Time: 4:00 Hours

[The figures in the right hand margin indicate full marks. Answer any **Three** questions from **Section-A** and any **Three** questions from **Section-B**]

Section - A

1. (a) What tasks are performed in the *front-end* and *back-end* of a compiler, and why? How are the *phases* related to the *passes*? (4)
- (b) How can immediate left-recursion be eliminated? Illustrate how left-recursion involving derivation of multiple steps can be eliminated using the following grammar: (3)

$$\begin{aligned} S &\rightarrow Pa \mid q \\ P &\rightarrow Pc \mid Sd \mid \epsilon \end{aligned}$$

- (c) Introduce the error recovery strategies in syntax analysis. (1.75)
2. Consider the following grammar:

$$S \rightarrow a S b S \mid b S a S \mid \epsilon$$

- (a) Compute FIRST and FOLLOW. (3)
- (b) Construct a predictive parsing table. (4)
- (c) Is the grammar LL(1)? (1.75)
3. (a) Consider the following grammar:

$$\begin{aligned} S &\rightarrow (L) \mid id \\ L &\rightarrow L, S \mid S \end{aligned}$$

- i. Construct the LR(0) items. (1.75)
- ii. Compute the LR(0) states. (5)
- (b) Differentiate between synthesized and inherited attributes. (2)
4. (a) Write a Yacc/Bison program that takes boolean expressions as input using following grammar and produces the truth value of the expressions. (5)

$$\begin{aligned} \text{bexpr} &\rightarrow \text{bexpr or bterm} \mid \text{bterm} \\ \text{bterm} &\rightarrow \text{bterm and bfactor} \mid \text{bfactor} \\ \text{bfactor} &\rightarrow \text{not bfactor} \mid (\text{bexpr}) \mid \text{true} \mid \text{false} \end{aligned}$$

- (b) What is the role of symbol table in compilers? Remark briefly on the key technical aspects in creating and managing the *symbol table*. (3.75)

Please go on to the next page...

Section - B

5. (a) When can a compiler's implementation of a language be called *strongly typed*? What is *widening* and *narrowing* in type conversion? (2)
- (b) Given an LR(k) item $[A \rightarrow \alpha \cdot \gamma, \delta]$ from a production $A \rightarrow \beta$, describe what A , α , γ and δ signify. (3)
- (c) Employing the following Action/Goto table (3.75)

State	Action		Goto	
	c	\$	S	T
0	Shift 1	Reduce $T \rightarrow \epsilon$	2	3
1	Shift 3	Reduce $S \rightarrow c$	0	
2		Accept	3	0
3	Shift 1	Accept		1

demonstrate the operation of a shift-reduce parser with the string "c". Show the stack contents, input and action at each step (assuming state 0 initially).

6. (a) Consider the following program in 3-address intermediate code

```

(1) a = 2
(2) b = 3
(3) t = a
(4) r1 = 0
(5) if (t ≤ 0) goto (9)
(6) t1 = 2 * a
(7) r1 = t1 + r1
(8) goto (5)
(9) t = b
(10) r2 = 0
(11) if (t ≤ 0) goto (15)
(12) t2 = 7 * b
(13) r2 = t2 + r2
(14) goto (11)
(15) t3 = r1 + r2
(16) r = 8 * t3

```

```

(1) t8 = j - 1
(2) t9 = 4 * t8
(3) tmp = A[t9]
(4) t10 = j + 1
(5) t11 = t10 - 1
(6) t12 = 4 * t11
(7) t13 = A[t12]
(8) t14 = j - 1
(9) t15 = 4 * t14
(10) A[t15] = t13
(11) t16 = j + 1
(12) t17 = t16 - 1
(13) t18 = 4 * t17
(14) A[t18] = tmp

```

- i. Indicate where new basic blocks start. For each basic block, give the line number such that the instruction in the line is the first one of that block. (3)
- ii. Give names $B_1, B_2, \dots$ for the program's basic blocks in the order the blocks appear in the given listing. Draw the control flow graph making use of those names. (3.75)
- (b) Construct the DAG for the basic block (2)

```

(1) d = b * c
(2) e = a + b
(3) b = b * c
(4) a = e - d

```

Please go on to the next page...

7. (a) What is the difference between a parse tree and an abstract syntax tree (AST)? (2)
(b) Introduce the idea of semantics preserving (local) optimizations in code optimization. (3)
(c) Illustrate how *triples*, *quadruples* and *static single assignment (SSA)* form are different in representing intermediate codes. (3.75)

8. Consider the following merge-sort program.

```
void mergesort(int a[], int i, int j)
{
    int mid;
    if (i < j)
    {
        mid = (i+j)/2;
        mergesort(a, i, mid); // left recursion
        mergesort(a, mid+1, j); // right recursion
        merge(a, i, mid, j); // merging of two sorted sub-arrays
    }
}

void merge(int a[], int p, int q, int r)
{
    /* Combine the elements back in a[p . . r] by merging
       the two sorted subarrays a[p . . q] and
       a[q+1 . . r] into a sorted sequence. */
    . . .
}

main()
{
    int a[11] = { 10, 14, 19, 26, 27, 31, 33, 35, 42, 44, 0 };
    mergesort(a, 0, 10);
}
```

- (a) Define following terms: activation tree and activation record. (2)
(b) Show the complete activation tree for the given *merge-sort* program. (3)
(c) Draw the activation records for the given *merge-sort* program. (3.75)

End of exam

Department of Computer Science & Engineering, University of Chittagong
7th Semester BSc Engineering Special Examination 2018
CSE 711: Compilers Marks: 52.5 Time: 4 hours

Using section-wise separate answer scripts, answer any three questions from each of the following two sections. Keep your handwriting legible.

Section A

1. (a) In the context of compiler construction, what do the terms *analysis* and *synthesis* signify? How do these phases differ in their objectives and operations, and in what ways do they complement each other within the compilation process?
- (b) Pertaining to the principles of programming languages, clarify the following two distinctions: i) *environment* and *state*, ii) *dynamic* and *static*.
- (c) How do the roles and responsibilities of a compiler's *front-end* differ from those of its *back-end*? Additionally, how do the concepts of *phases* and *passes* differ in compiler design, and what is the significance of this distinction? [3+2.75+3]
2. (a) Introduce the notions of *token*, *pattern* and *lexeme*. Point out how they are related.
- (b) Remark briefly on the key technical aspects in creating and managing the *symbol table*.
- (c) What are synthesized and inherited attributes? What are S-attributed and L-attributed grammars? [3+3+2.75]
3. Consider the following grammar

$$S \rightarrow a S b S \mid b S a S \mid \epsilon$$

- (a) Compute FIRST and FOLLOW.
- (b) Construct a predictive parsing table.
- (c) Is the grammar LL(1)? [3+4+1.75]
4. Consider the following grammar

$$S \rightarrow (L) \mid id$$

$$L \rightarrow L , S \mid S$$

- (a) Construct the SLR sets of items for the grammar.
- (b) Show the PDA or transition table for recognizing viable prefixes of this grammar.
- (c) Compute GOTO function for these sets of items. [2.75+3+3]

Section B

1. (a) Consider the following grammar and associated parsing table.

	State	a	b	c	\$	\$	X
0: $S \rightarrow X$	1	s2	s3			g5	g4
1: $X \rightarrow a X c$	2	s2	s3				g6
2: $X \rightarrow X X$	3	r3	r3	r3	r3		.
3: $X \rightarrow b$	4	s2	s3		r0		g7
	5				Acpt		
	6		s3	s8			g7
	7	r2	r2	r2	r2		
	8	r1	r1	r1	r1		

Show the execution of the parser on the string *abb*.

(b) Translate the arithmetic expression $a = b[i] + c[i]$ into: i) a syntax tree ii) quadruples, triples and iv) indirect triples. [4+4]

6. (a) What is a "shift" in a bottom-up parser? What is a "reduce" in a bottom-up parser?

(b) Construct the *directed acyclic graph* (DAG) for the following expression, and then show its corresponding three-address code
 $w + w * (x - y) + (x - y) * z$

(c) Give annotated parse trees for the inputs $9 - 5 + 2$ and $9 - 5 * 2$. [3+3+2.75]

7. (a) What is the difference between a parse tree and an abstract syntax tree (AST)?

(b) Why compilers use intermediate representations instead of translating directly from source to target language?

(c) What role does the runtime stack, particularly through activation records, play in organizing memory during program execution? What are the key concerns that influence efficient heap management at runtime? [3 + 2.75 + 3]

8. (a) Explain why and how programs exhibit *spatial* and *temporal locality* during execution.

(b) Construct the basic blocks from the following intermediate code and show the associated flow graph.

```
1) t1 = p + q
2) t2 = t1 - a
3) x = t2
4) t3 = z > 0
5) if t3 goto (10)
6) t4 = z + 1
7) x = t4
8) t5 = p + q
9) goto (4)
10) m = t5
```

[2.75+6]

University of Chittagong
Department of Computer Science & Engineering
Seventh Semester B.Sc. (Engg.) Final Examination, 2017
Course code: CSE 711
Course Title: Compilers
Full Marks: 52.5, Duration: 4 Hours

(Answer any six (6) questions taking at least three (3) from section A and (3) from section B)
(Separate answer scripts must be used for section-A and Section-B)
Figures in the right margin indicate marks of each question

SECTION A

- 1.a What do *environment* and *state* mappings mean? Can there be cases of static bindings in this regard? 2
- b How is compiler technology affected by the followings: safety oriented high level source languages (e.g., Java), RISC/CISC as well as multi-threaded/multicore architectures of target machines? 3
- c What are the typical phases in a compiler? What are the advantages in splitting compilation into multiple phases? How are the phases related to analysis and synthesis? 3.75
- 2.a Briefly describe different general strategies that a parser can employ to recover from a syntactic error. 2
- b How is a token different from a lexeme? How does a pair of input buffers help in scanning? 3
- c How can immediate left-recursion be eliminated? Illustrate how left-recursion involving derivation of multiple steps can be eliminated using the following grammar: 3.75

$S \rightarrow Pa | q$
 $P \rightarrow Pc | Sd | \epsilon$

3. Consider the following grammar:
 $S \rightarrow Sb | Sa | a$
 - a Explain why predictive parsing cannot work with this grammar. Apply appropriate transformation to the above grammar for predictive parsing to work. 2
 - b Build the LL(1) parsing table for the transformed grammar using FIRST and FOLLOW. 3
 - c Using the LL(1) table constructed, show how a table-driven parser parses the string abab. 3.75
- 4.a With regard to type checking, what do you know about *type-expression* and *type-conversion*? 2
- b How can the following statement be translated into intermediate code employing *short circuit*? 3

if ($y < 90 \parallel y > 100 \&\& y \neq z$) $x = 1$;

- c Consider the following attribute grammar for calculating values of signed binary numbers. 3.75

PRODUCTION	SEMANTIC/ATTRIBUTE RULES
$\text{Number} \rightarrow \text{Sign List}$	$\text{List.pos} = 0$ If Sign.neg then $\text{Number.val} = - \text{List.val}$ else $\text{Number.val} = \text{List.val}$
$\text{Sign} \rightarrow +$	$\text{Sign.neg} = \text{false}$
$\text{Sign} \rightarrow -$	$\text{Sign.neg} = \text{true}$
$\text{List}_0 \rightarrow \text{List}_1 \text{ Bit}$	$\text{List}_1.\text{pos} = \text{List}_0.\text{pos} + 1$ $\text{Bit.pos} = \text{List}_1.\text{pos}$ $\text{List}_0.\text{val} = \text{List}_1.\text{val} + \text{Bit.val}$
$\text{List}_0 \rightarrow \text{Bit}$	$\text{Bit.pos} = \text{List}_0.\text{pos}$ $\text{List}_0.\text{val} = \text{Bit.val}$
$\text{Bit} \rightarrow 0$	$\text{Bit.val} = 0$
$\text{Bit} \rightarrow 1$	$\text{Bit.val} = 2^{\text{Bit.pos}}$

List the attributes that are inherited and those that are synthesized. Is this definition *L-attributed*?

SECTION B

- 5.a Which parsing method(s) you have studied can parse strings in the following grammar? And which methods cannot? Explain why in a few sentences. 2

$$\begin{aligned} E &\rightarrow A \mid B \\ A &\rightarrow a \mid c \\ B &\rightarrow b \mid c \end{aligned}$$

- b Determine FIRST and FOLLOW sets for the following grammar: 3

$$\begin{aligned} S &\rightarrow (XSS) \mid (XS) \mid a \\ X &\rightarrow b \end{aligned}$$

- c Is the above grammar SLR(1)? Why or why not? 3.75

- 6.a What are the desirable properties in a memory manager when it comes to managing the heap? 2

- b For the following statements pertaining to LR parsing, fill in the blanks with accurate clauses/phrases: 3

$$[A \rightarrow \alpha \cdot \delta, \delta]$$

$$A \in [A \rightarrow \alpha \cdot \delta, \delta]$$

Given an LR (k) item $[A \rightarrow \cdot]$, the LR (k) items that must be added to the closure are If the parser is in a state containing an LR (k) item $[A \rightarrow \cdot]$, the top of the stack must contain A shift/reduce conflict occurs when The reason why there are no shift/shift conflict is ...

- c Describe quadruple, triples and indirect triples representation of three address code. 3.75

- 7.a Construct the directed acyclic graph for the following expression: 2

$$((a - b) + ((a + b) * (a - b))) - ((a + b) * (a - b))$$

- b What are called optimizations and what is an optimization compiler? 2

- c Construct the basic blocks from the following intermediate code and show the associated flow graph. 3

$$1) y = k - 1$$

$$2) t1 = x + y$$

Department of Computer Science and Engineering
University of Chittagong
7th Semester BSc (Engg.) Examination 2016
CSE 711: COMPILERS
Marks: 52.5 Time: 4 hours

[Answer any three questions from each of the following sections. All subparts of a question should be answered chronologically]

Section A

- 1 (a) In constructing a compiler, why/how would you possibly find its decomposition into *front-end* and *back-end* useful? In the same vein, how would grouping of *phases* and multiple *passes* be convenient? 3.5
- (b) Consider the state-of-the-art in computing pertaining to increasingly larger sizes of input data, multi-core architectures and levels of cache memories. In this backdrop, how does compiler technology relates to transforming high-level source programs into efficiently executing programs? 3.25
- (c) As separate tasks from that of a *compiler proper*, state the roles of *preprocessor*, *assembler*, *linker* and *loader*. 2
- 2 (a) State, preferably with illustrative examples, the connections and the distinctions among the notions *token*, *pattern* and *lexeme*. 2.25
- (b) What is ambiguity in grammar? How does it relate to the rules of associativity and precedence from the standpoint of writing a compiler? 2.5
- (c) For the syntax-directed definition given below 2+2

PRODUCTION	SEMANTIC RULES
$F \rightarrow D$	$F.w = 1$ $F.v = D.v$ $D.s = -1$
$F \rightarrow F_1 D$	$F.w = F_1.w + 1$ $F.v = F_1.v + D.v$ $D.s = -F_1.w - 1$
$D \rightarrow 0$	$D.v = 0$
$D \rightarrow 1$	$D.v = 2^{D.s}$

determine the attributes that are *inherited* and those that are *synthesized*, and comment on whether this syntax-directed definition is *L-attributed* or not.

- 3 (a) Consider a grammar with nonterminals S and T and the following productions: 3+2.75

$$S \rightarrow bSa \mid bT$$

$$T \rightarrow Td \mid c$$

where S is the start symbol and a, b, c and d are terminals. What issue(s) makes this grammar unfit for writing a *recursive descent* parser with a single token lookahead? Get rid of the issue(s), and write a *recursive descent* parser for the modified grammar (Pseudocode without declarations would do; Use function `scan()` to read the next input token and `err()` to report errors, if needed.)

(b) Calculate the cost of the given instruction sequence.

MOV b, R0

ADD c, R0

MOV R0, a

4 (a) What does LL(k) signify? What are the pros and cons in LL(k) parsing with larger values of k?

1+1.75

(b) Under what conditions does the CYK algorithm work? Illustrate how to employ the CYK algorithm to decide the membership (or lack thereof) of the string 111000 in the language described by the following grammar:

$S \rightarrow \epsilon | NZ | XZ$

$R \rightarrow NZ | XZ$

$X \rightarrow NR$

$N \rightarrow 1$

$Z \rightarrow 0$

6
(0.5+3.5
+0.5+1.5)

Comment on whether the CYK algorithm is only a recognizer (deciding membership of strings to the language) or it is essentially a parser as well. Identify the main reason(s) why a CYK algorithm based parsing mechanism might not turn out to be practical/efficient.

Section B

5 (a) Employing the following Action/Goto table

State	Action		Goto	
	c	\$	S	T
0	Shift 1	Reduce $T \rightarrow \epsilon$	2	3
1	Shift 3	Reduce $S \rightarrow \epsilon$	0	
2		Accept	3	0
3	Shift 1	Accept		1

demonstrate the operation of a shift-reduce parser with the string "c". Show the stack contents, input and action at each step (assuming state 0 initially).

3

(b) Given an LR(k) item $[A \rightarrow \alpha \cdot \gamma, \delta]$ from a production $A \rightarrow \beta$, describe what A , α , γ and δ signify.

3

(c) When can a compiler's implementation of a language be called *strongly typed*? What is *widening* and *narrowing* in type conversion?

1.25+1.5

6 (a) Comment on the possible conflicts in LR parsing. Compare, in brief, SLR(1) and canonical LR(1) with regard to ease of implementation and parsing power.

1.5+2.25

(b) For the following (augmented) grammar (with terminals 0 and 1)

$S' \rightarrow S$

$S \rightarrow 0S1 | 0$

compute the canonical set of LR(1) items showing the corresponding DFA. Also construct the LR(1) Action/Goto table.

3.5+1.5

page 2 of 4

Construct the DAG for the following expression, and then show its corresponding three-address code 2.75

$$((x - y) + ((x - y)/(x + y))) - ((x - y)/(x + y))$$

- (b) Construct the basic blocks from the following intermediate code and show the associated flow graph. 3

```
1) t1 = a + b
2) t2 = t1 - c
3) x = t2
4) t3 = x > 0
5) if t3 goto (10)
6) t4 = x + 1
7) x = t4
8) t5 = a + b
9) goto (4)
10) m = t5
```

- (c) Consider the following skeletal code snippet in an operating system source code

```
cpu_lock = 1;
... code for critical task ...
cpu_lock = 0;
```

where `cpu_lock` seeks to prevent interrupt while the critical code gets executed. Now that there are a number of semantics-preserving transformations in which a compiler seeks to improve a program in general, some transformation, if applied to the above OS code, can possibly result in a problematic situation. Identify what that transformation is, illustrate how it works in usual settings, and state why/how it would be troublesome in this case. 3

- 8 (a) How is the memory of a running program typically subdivided? What are the forms of *locality* exhibited by programs during execution? 1.75+1
- (b) Show the diagrams of *activation records* (stack frame) at *point one* and *point two* indicated in the following code: 3

```
#include <stdio.h>

int A(int a)
{
    int r;
    r = a * 10;
    // point one
    return r;
}

int B(int b)
{
    int s;
    s = b % 8;
    // point two
    return s;
}
```



```

int C(int c1, int c2)
{
    int v1, v2;
    v1 = A(c2);
    v2 = B(c1);
    return v1 + v2;
}

```

```

int main(void)
{
    int m = 4, n = 31, p = 10;
    p = p + C(n + 3, m + 1);
    printf("p = %d\n", p);
    return 0;
}

```

- (c) From the given program, show the three address statement performing this computation.

3

```

begin
    prod:=0;
    i:=1;
    do begin
        prod:=prod+a[i]*b[i];
        i:=i+1
    end
    while i<=20
end

```